Document number: P3330R0
Date: 2024-6-17.
Authors: Gonzalo Brito Gadeschi, Damien Lebrun-Grandie.
Reply to: Gonzalo Brito Gadeschi <gonzalob _at_ nvidia.com (http://nvidia.com)>.
Audience: SG1.

Table of Contents

User-defined Atomic Read-Modify-Write Operations

We propose extending the read-modify-write APIs of `std::atomic` and `std::atomic_ref` with user-defined read-modify-write atomic operations. These move the responsibility of writing CAS-like loops from user code into the implementation, improving the portability, performance, and forward-progress of C++ applications.

Motivation

Without these extensions, users are required to implement user-defined read-modify-write operations with `compare_exchange` loops (CAS-loops from now on), which:

- are hard to write correctly without compromising the QoI of forward progress,
- complicate compiler optimizations (no compiler optimizes a CAS-loop that does an `add` into an atomic read-modify-write `fetch_add`), and
- do not perform well for non-lockfree atomics since CAS-loops take and release the lock multiple times per iteration.

Design

> **Note**: a portable reference implementation is provided here (https://gcc.godbolt.org/z/jvo8jEfzj) for exposition purposes.

We propose extending `std::atomic` and `std::atomic_ref` with user-defined read-modify-write atomic operations:

```
template <typename T, typename UnaryOp>
bool fetch_update(T& old, UnaryOp uop,
                  memory_order order = memory_order::seq_cst) const noexcept;

template <typename T, typename UnaryOp>
bool fetch_update(T& old, UnaryOp uop,
                  memory_order success = memory_order::seq_cst,
                  memory_order failure = memory_order::seq_cst) const noexcept;
```

The result of the unary operation `uop` is used to atomically update the value refered by the atomic type and it may be of type `T` or `optional<T>`. If `uop` returns:

- a value of type `T`, or of type `optional<T>` with `has_value()` true, then `fetch_update` writes the value returned by `uop` to memory and returns `true` to indicate its effect is that of an atomic read-modify-write operation,
- `nullopt`, then `fetch_update` returns `false` to indicate its effect is that of an atomic read operation.

In both cases, the value fetched by `fetch_update` and passed to `uop` is written to `old`.

This API is similar to that of `compare_exchange` operations and enables exposing vendor-specific atomic operations (e.g., `fetch_fp_exp`):

Before	After

`#include <atomic>`
`atomic<float> a = 0.f;`
 	
`int main() {`
`  float old = a.load(), next;`
`  do {`
`  next = std::fexp(old, 2.f);`
`  } while(a.compare_exchange_strong(old, next));`
`  return 0;`
`}`
	`#include <atom`
	`atomic<float>`
	
	`int main() {`
	`  float old;`
	`  a.fetch_upda`
	`  return s`
	`  });`
	`  return 0;`
	`}`

The following example shows how to use the API to update a `pair` depending on the value of one of its sub-objects, aborting the update and turning the operation into an atomic read when a certain condition is met:

```
atomic<pair<char, short>> atom;

pair<char, short> old;
bool success
  = atom.fetch_update(old, [](pair<char, short> p) -> optional<par<char, sho
    if (p.first > 42) return nullopt;
    return make_pair(p.first+1, p.second+2)};
});
assert((success && old.first <= 42) || (!success && old.first > 42));
```

Implementation strategies

The semantics this API may provide are restricted by the implementation strategies allowed and vice-versa. This design chooses to support, among others, following implementation strategies:

1. **CAS-loop** based implementation using atomic CAS operations.
2. **Lock** based implementations that take the lock:
    - Multiple-times (e.g. lock-based CAS-loop).
    - Only once.
3. **LL/SC** based implementations.

4. **Hardware Transactional Memory** (HTM) based implementations.

The design also aims at simplifying the implementation requirements for leveraging hardware acceleration.

Requirements on the user-defined operation

To support these implementation strategies, the `UnaryOp(T)` call-operator and `uop(old)` call-expression are required to:

- satisfy `regular_invocable` (for same inputs, produces same output), and
- be an implicit-lifetime type, and
- `noexcept(declval<UnaryOp>()(declval<T>()))` is true, and,
- only accesses its operands or its non-static data-members, and
- not perform a library I/O function call, a synchronization operation, or an atomic operation, and
- eventually return.

This requirements should imply that:

- the only side-effects `uop(old)` may have are modifying its non-static data members,
- for the same inputs it always produces the same outputs, independetly of whether other state of the program has changed because `uop(old)` can't access such state.

Requirements on the implementation

The effect of `fetch_update` is to call `uop(old)` *exactly once*. Implementation strategies that call `uop(old)` multiple times are allowed as long as they behave "as if" `uop(old)` was called exactly once.

For example, CAS-loop based implementations must use that `UnaryOp` is implicit-lifetime to back up the `uop` object, and call `uop(old)` on a fresh copy of the backup every iteration, to discard any modifications that `uop(old)` may have done to the `uop` object itself.

Limitations for users

The major downsides of these requirements are that:

- they prevent the use of `printf` -based debugging within `UnaryOp` ,

Do the requirements protect from ABA (https://en.wikipedia.org/wiki/ABA_problem)? (TODO: add an example showing that they do not).

Alternatives

Does restricting `UnaryOp` to be `constexpr` suffice?

No, `constexpr` functions in C++26 may not return in a finite set of steps, e.g., on free-standing implementations that do not replace the empty iteration statement in `while(1);` with a call to the non-`constexpr` API `std::this_thread::yield()`, enabling `while(1)` within constant evaluation:

```
constexpr void hang() { while(1); }
```

Furthermore, P3309 (http://wg21.link/P3309) proposes `constexpr` atomics and synchronization operations.

Notes on Forward progress

Notice that relaxing `UnaryOp` constraints to:

- allowing `UnaryOp` to write to memory outside the `UnaryOp` itself, and
- calling synchronization functions (like atomics),

would make the following program legal:

```
// Litmus test 0: discovered concurrency
atomic<int> atom, counter = 0;
static_assert(atomic<int>::is_lockfree);

int main() {
  std::jthread t0([&] { atom.fetch_update([&](int) {
    counter++;
    while(counter.load() != 2);
    return 0;
  })});

  std::jthread t1([&] { atom.fetch_update([&](int) {
    counter++;
    while(counter.load() != 2);
    return 0;
  })});

  return 0;
}
```

If the specification would require that this program terminates, then the following implementations become invalid for this execution:

1. Taking a single lock for the whole duration of `fetch_update`:
   - Such an implementation deadlocks for the execution above.
2. Hardware transactional memory (HTM):
   - Such an implementation does not make progress: as transaction time out is exceeded, the transaction is reverted, including the counter++ update, before yielding to the other thread.

It would not prevent a lock-based implementation that uses a CAS-loop, since an implementation that takes the lock multiple times within `fetch_update` makes progress.

The restrictions above preserve these two implementation opportunities.

Portable CAS-loop

The proposed `fetch_update` semantics could be weakened by removing *all* restrictions on `UnaryOp`, since atomic read-modify-write operations are atomic w.r.t. the modification order of the memory location being operated on. This is the implementation pursued by, e.g., Rust's atomic fetch_update (https://doc.rust-lang.org/std/sync/atomic/struct.AtomicUsize.html#method.fetch_update) APIs.

Then *concurrent forward progress* guarantees that Litmust test 0 (discovered concurrency) terminates, which implementations that take a single-lock for the whole `fetch_update` or use HTM fail to uphold in that case.

That does not mean that single-lock or HTM cannot be used, but rather that compiler analysis is required to prove that `UnaryOp` does not perform any of the operations that would enable those implementations to be valid (if the compiler can prove that, then they can still be used).

The proposed semantics are forward compatible with this weakening, i.e., it can be done later without breaking any valid code.

Transactional guarantees

The proposed `fetch_update` semantics could be strengthened by guaranteeing that every `UnaryOp` call happens-before another `fetch_update` call as part of an atomic access to the same memory location.

This would require implementing `fetch_update` as an *atomic transaction* (e.g. as per P1875: Transactional Memory Lite (https://open-std.org/JTC1/SC22/WG21/docs/papers/2021/p1875r2.pdf)), and would guarantee the absence of data-races in the following litmus test:

```
// Litmus Test 1: atomic transaction
atomic<int> atom;
int counter = 0;

int main() {
  std::jthread t0([&] { atom.fetch_update([&](int) {
    counter++;
    return 0;
  })});

  std::jthread t1([&] { atom.fetch_update([&](int) {
    counter++;
    return 0;
  })});
  return 0;
}
```

This strengthening would require implementing `fetch_update`:

- with a single lock for the whole `fetch_update`, or
- hardware transactional memory.

The proposed semantics are forward compatible with this strengthening, i.e., it can be done later without breaking any valid code.

`std::expected`-based API

An alternative API choice is to use `std::expected` to return the value in the success or failure case:

```
template <typename UnaryOp>
expected<T, T> fetch_update(UnaryOp uop,
               memory_order order = memory_order::seq_cst) const noexcept;

atomic<pair<char, short>> atom;

auto r = atom.fetch_update([](pair<char, short> p) -> optional<pair<char,
short>> {
    if (p.first > 42) return nullopt;
    return make_pair(p.first+1, p.second+2)};
});
if (r) assert(r.value().first <= 42);
else assert(r.error().first > 42);
```

Function object return type

The function object return type is mandated to be `T` or `optional<T>`.

Alternatives considered:

- `UnaryOp(T, bool&) -> T` and signal failure through `bool&`: requires constructing a `T` dead value for the return.
- `UnaryOp(T) -> T` and signal failure by throwing an exception: no.
- Don't support failure: e.g. by returning the same value as passed in, would either perform an extra write, or whether an atomic read-modify-write vs atomic read happens would be "implicit" depending on whether the value `memcmp` 's the same.

Prior art

Rust's fetch_update (https://doc.rust-lang.org/std/sync/atomic/struct.AtomicUsize.html#method.fetch_update). Kokkos uses a similar API internally.

Wording

Add feature test macro to `<version>` synopsis:

```
#define __cpp_lib_atomic_fetch_update 202XXXL // also in <atomic>
```

For each of the following specializations [atomics.ref.int], [atomics.ref.float], [atomics.types.int], [atomics.types.float], and the generic [atomics.ref.generic.general] and [atomics.types.generic.general], add the following public function after their last public function:

> ```
> ...
> template <typename T, typename UnaryOp>
> bool fetch_update(T& old, UnaryOp uop,
>                   memory_order order = memory_order::seq_cst) const noexcept
> template <typename T, typename UnaryOp>
> bool fetch_update(T& old, UnaryOp uop,
>                   memory_order success = memory_order::seq_cst,
>                   memory_order failure = memory_order::seq_cst) const noexce
> ...
> ```

- *Mandates*: `regular_invocable<UnaryOp, T>` is true, `noexcept(uop(declval<T>()))` is true, and either `is_same_v<invoke_result_t<UnaryOp, T>, T>` or `is_same_v<invoke_result_t<UnaryOp, T>, optional<T>>` is true.

- *Preconditions*: `failure` is `memory_order::relaxed`, `memory_order::consume`, `memory_order::acquire`, or `memory_order::seq_cst`. The `UnaryOp` call operator selected by the `uop(old)` call expression:
  - only accesses its operand or the non-static data-members of `uop`, and
  - does not perform a library I/O function call, a synchronization operation, or an atomic operation, and
  - eventually returns [intro.progress] (https://eel.is/c++draft/intro.progress).

- *Effects*: Atomically:
  - retrieves the value referenced by `*ptr` into `old`, and
  - if `invoke_result_t<UnaryOp, T>` is `T`, replaces the value referenced by `*ptr` with the result of `uop(old)`, in which case memory is affected according to the value of `success`, or
  - `uop(old).has_value()` is `true`, replaces the value referenced by `*ptr` with the result of `uop(old).value()`, in which case memory is affected according to the value of `success`, or
  - memory is affected according to the value of `failure`.

  When only one `memory_order` argument is supplied, the value of `success` is `order`, and the value of `failure` is `order` except that a value of `memory_order::acq_rel` shall be replaced by the value `memory_order::acquire` and a value of `memory_order::release` shall be replaced by the value `memory_order::relaxed`.

- *Returns*: `true` if the operation replaced the value of `*ptr` and `false` otherwise.
  [Note: if `fetch_update` returns `true`, it is an atomic read-modify-write operation according to [intro.races], and an atomic read otherwise - end note]